LAB DAY 6

NAME: Noelshaun Deril.W

REG. NO.: 192324174

COURSE CODE AND NAME: CSA0617 DESIGN AND ANALYSIS OF ALGORITHM

BELLMAN-FORD ALGORITHM

PYTHON CODE:

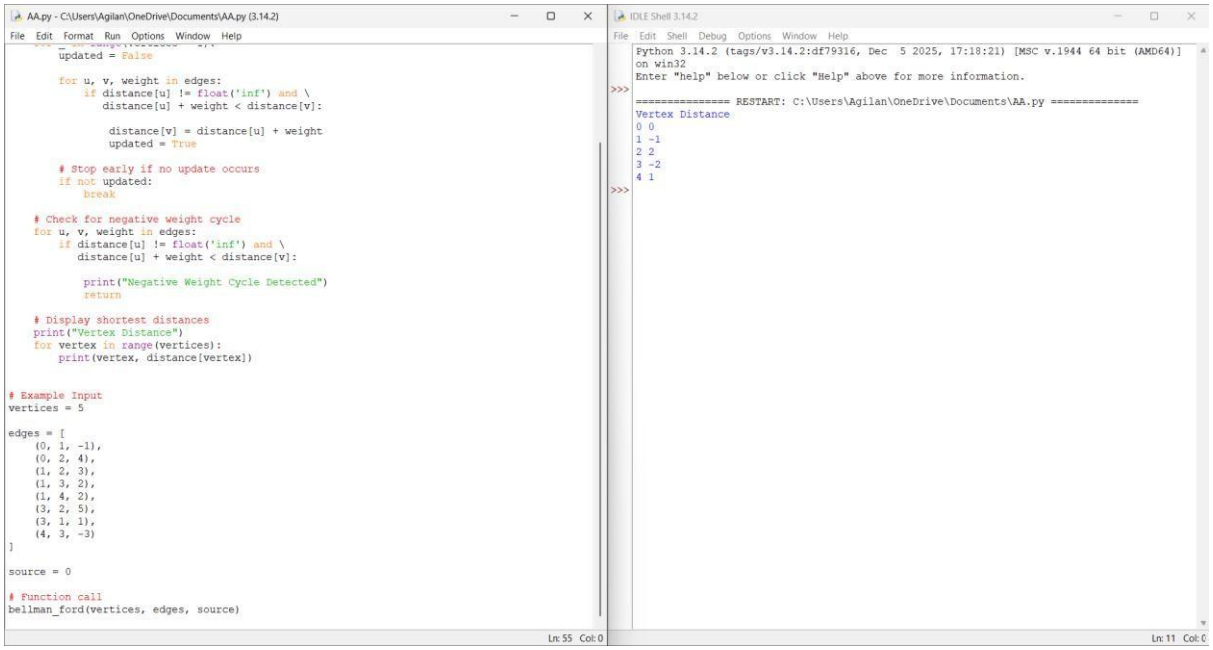
```
# Bellman-Ford Algorithm

def bellman_ford(vertices, edges, source):
    # Initialize distances     distance
= [float('inf')] * vertices
distance[source] = 0

    # Relax all edges V-1 times
for _ in range(vertices - 1):
        updated = False

        for u, v, weight in edges:
            if distance[u] != float('inf') and \
distance[u] + weight < distance[v]:

                distance[v] = distance[u] + weight
updated = True

        # Stop early if no update occurs
if not updated:
            break
```

```
    # Check for negative weight cycle
for u, v, weight in edges:
        if distance[u] != float('inf') and \
distance[u] + weight < distance[v]:

            print("Negative Weight Cycle Detected")
            return

    # Display shortest distances
print("Vertex Distance")     for
vertex in range(vertices):
print(vertex, distance[vertex])



# Example Input
vertices = 5

edges = [
    (0, 1, -1),
    (0, 2, 4),
    (1, 2, 3),
    (1, 3, 2),
    (1, 4, 2),
    (3, 2, 5),
    (3, 1, 1),
    (4, 3, -3)
]

source = 0
```

Function call bellman_ford(vertices,

edges, source)

EXECUTION & OUTPUT:



FLOYD-WARSHALL ALGORITHM

PYTHON CODE:

Floyd-Warshall Algorithm

INF = float('inf')

def floyd_warshall(graph):

n = len(graph)

Create a copy of the graph

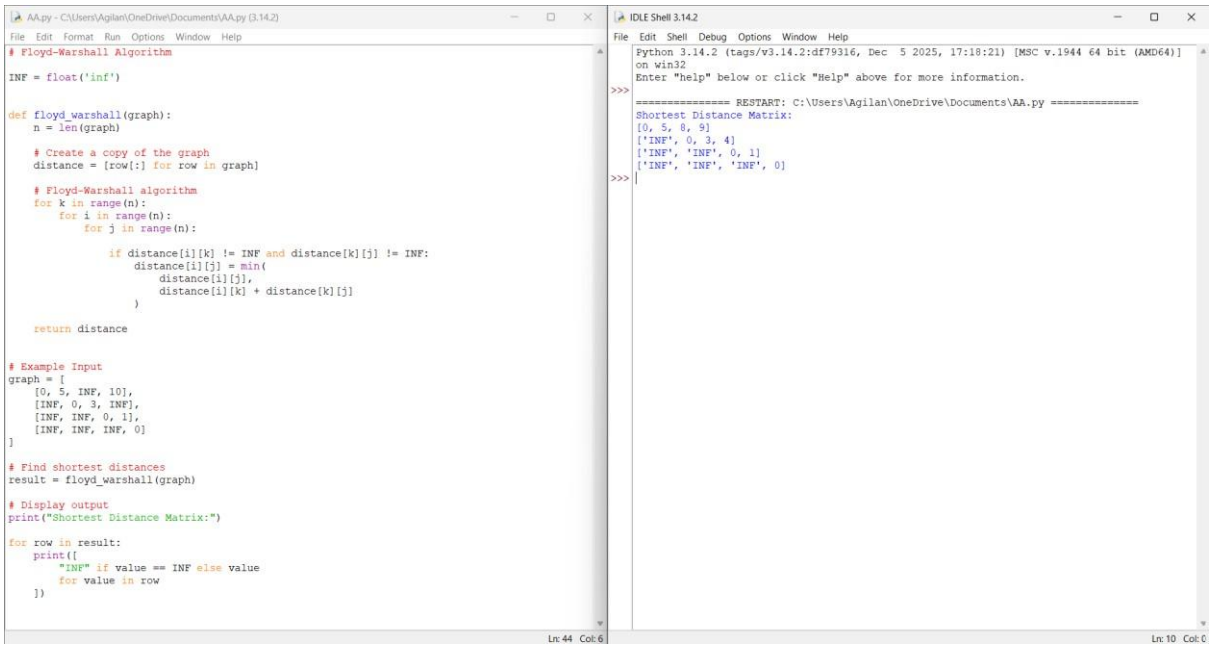
distance = [row[:] for row in graph]

Floyd-Warshall algorithm for k in

```
range(n):         for i in range(n):
for j in range(n):

                if distance[i][k] != INF and distance[k][j] != INF:
                    distance[i][j] = min(
distance[i][j],                         distance[i][k]
+ distance[k][j]
                    )

    return distance



# Example Input graph
= [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]

# Find shortest distances result
= floyd_warshall(graph)

# Display output print("Shortest
Distance Matrix:") for row in
result:

    print([
        "INF" if value == INF else value
for value in row
```

])

EXECUTION & OUTPUT:



WARSHALL's ALGORITHM

PYTHON CODE:

Warshall's Algorithm

def warshall(graph):

n = len(graph)

Create a copy of the graph

reach = [row[:] for row in graph]

Warshall's Algorithm

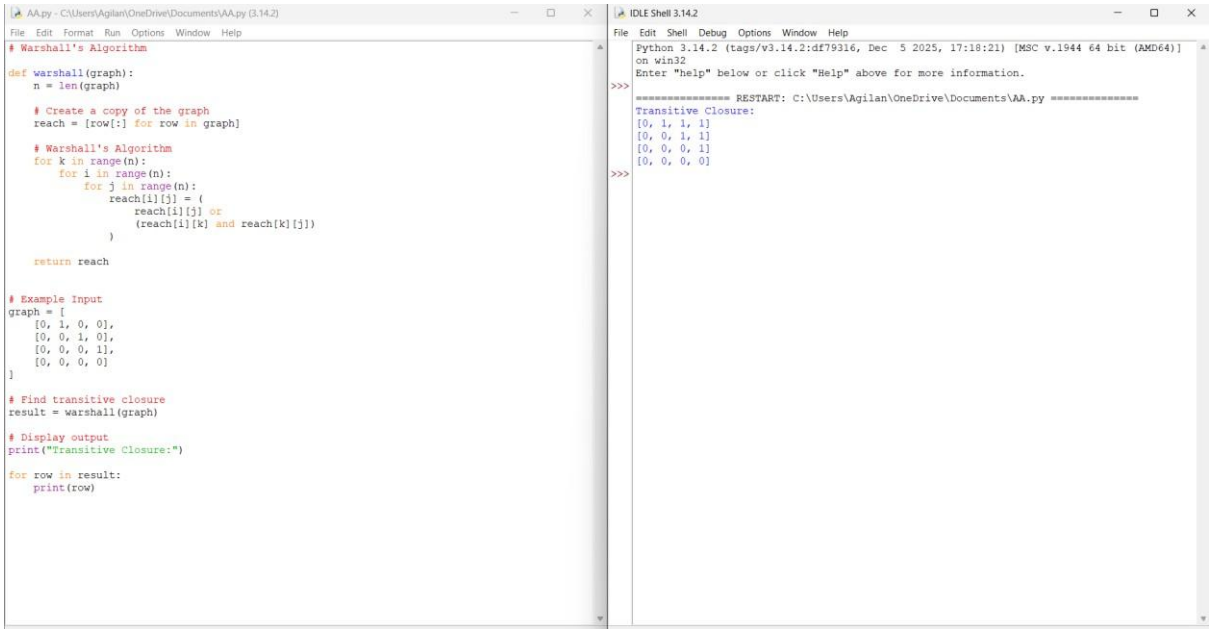
for k in range(n):

for i in range(n):

for j in range(n):

```
reach[i][j] = (
reach[i][j] or
                (reach[i][k] and reach[k][j])
            )

    return reach



# Example Input graph
= [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

# Find transitive closure result
= warshall(graph)

# Display output print("Transitive
Closure:")

for row in result:
print(row)
```

EXECUTION & OUTPUT:



FIND THE CITY WITH THE SMALLEST NUMBER OF REACHABLE CITIES

PYTHON CODE:

Find the City with the Smallest Number of Reachable Cities

Using Floyd-Warshall Algorithm

INF = float('inf')

def find_city(n, edges, threshold):

Create distance matrix dist =

[[INF] * n for _ in range(n)]

Distance from a city to itself is 0

for i in range(n): dist[i][i] = 0

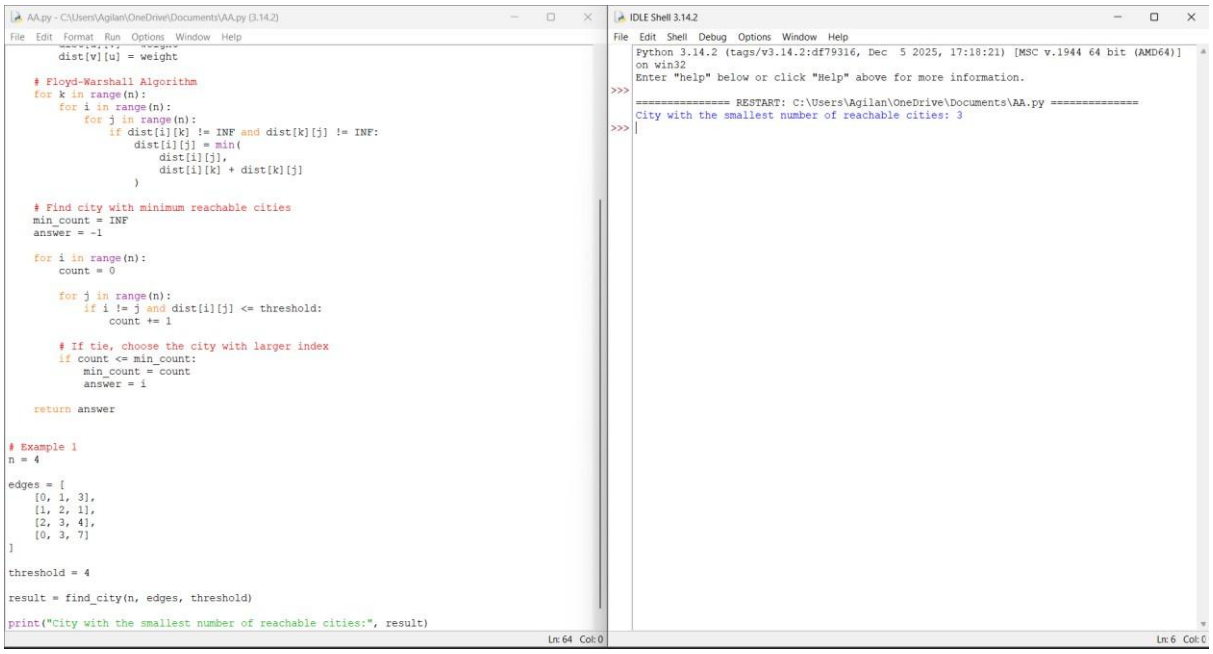
Add edges for u, v,

weight in edges:

```
dist[u][v] = weight
dist[v][u] = weight

    # Floyd-Warshall Algorithm      for k in
range(n):          for i in range(n):              for j in
range(n):                  if dist[i][k] != INF and
dist[k][j] != INF:
                    dist[i][j] = min(
dist[i][j],                         dist[i][k]
+ dist[k][j]
                    )

    # Find city with minimum reachable cities
min_count = INF      answer = -1

    for i in range(n):
count = 0

        for j in range(n):              if i != j and
dist[i][j] <= threshold:
                count += 1

        # If tie, choose the city with larger index
if count <= min_count:              min_count =
count              answer = i

    return answer
```

```
# Example 1 n
= 4


edges = [
    [0, 1, 3],
    [1, 2, 1],
    [2, 3, 4],
    [0, 3, 7]
]


threshold = 4


result = find_city(n, edges, threshold)


print("City with the smallest number of reachable cities:", result)
```

EXECUTION & OUTPUT:



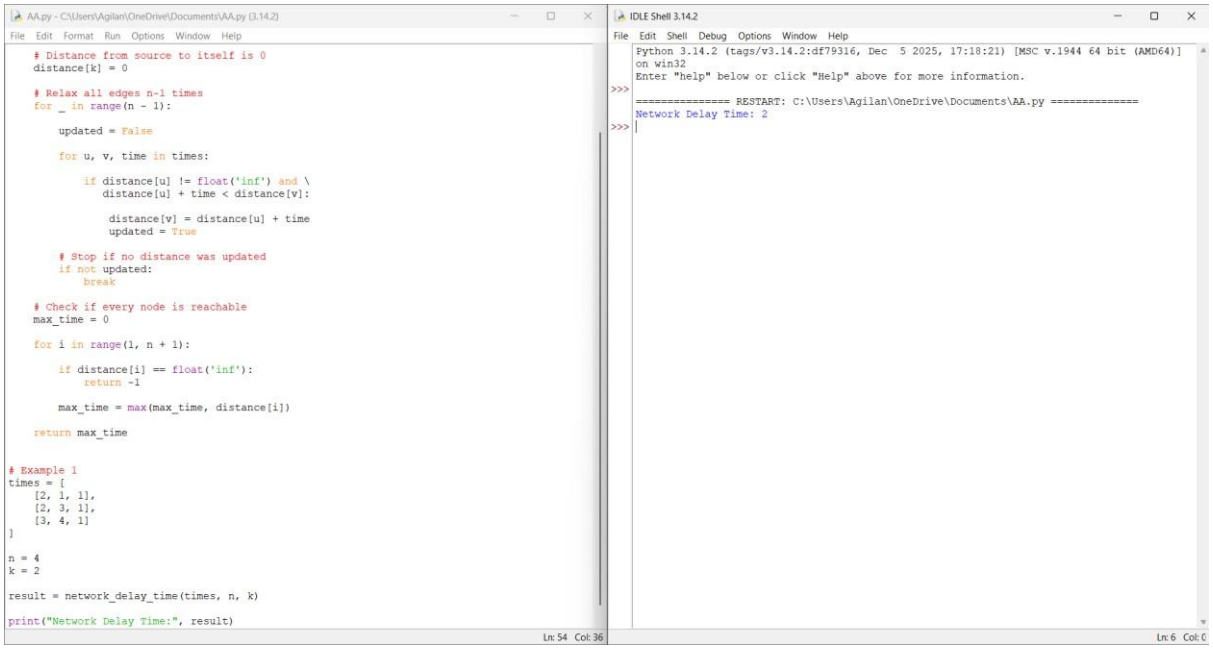
NETWORK DELAY TIME

PYTHON CODE:

```
# Network Delay Time
# Using Bellman-Ford Algorithm

def network_delay_time(times, n, k):

    # Initialize distances     distance
= [float('inf')] * (n + 1)

    # Distance from source to itself is 0
distance[k] = 0

    # Relax all edges n-1 times
for _ in range(n - 1):

        updated = False

        for u, v, time in times:

            if distance[u] != float('inf') and \
distance[u] + time < distance[v]:

                distance[v] = distance[u] + time
updated = True

        # Stop if no distance was updated
if not updated:
            break
```

```
    # Check if every node is reachable
max_time = 0

    for i in range(1, n + 1):

        if distance[i] == float('inf'):
            return -1

        max_time = max(max_time, distance[i])

    return max_time



# Example 1 times
= [
    [2, 1, 1],
    [2, 3, 1],
    [3, 4, 1]
]

n = 4
k = 2

result = network_delay_time(times, n, k)

print("Network Delay Time:", result)
```

EXECUTION & OUTPUT:



CHEAPEST FIGHT WITHIN K STOPS

PYTHON CODE:

```
# Cheapest Flight Within K Stops
# Using Bellman-Ford Algorithm

def cheapest_flight(n, flights, src, dst, k):

    # Initialize distances
distance = [float('inf')] * n
distance[src] = 0

    # Maximum number of edges = K + 1
for _ in range(k + 1):

        # Make a copy so that only one level of flight
# is considered in each iteration          temp =
distance[:]
```

```
        for u, v, cost in flights:

            if distance[u] != float('inf'):

                new_cost = distance[u] + cost

                if new_cost < temp[v]:
temp[v] = new_cost

        distance = temp

    # If destination cannot be reached
if distance[dst] == float('inf'):
        return -1

    return distance[dst]


# Example 1 n
= 4

flights = [
    [0, 1, 100],
    [1, 2, 100],
    [2, 3, 100],
    [0, 3, 500]
]
```

src = 0

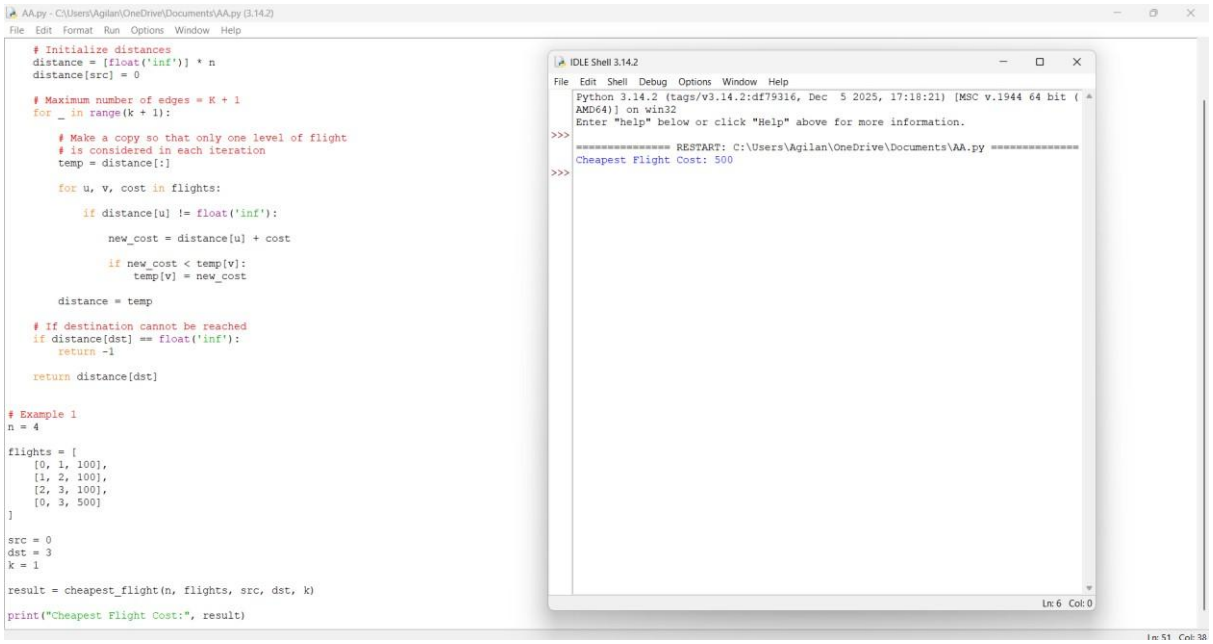
dst = 3 k

= 1

result = cheapest_flight(n, flights, src, dst, k)

print("Cheapest Flight Cost:", result)

EXECUTION & OUTPUT:



DETECT NEGATIVE WEIGHT CYCLE

PYTHON CODE:

Detect Negative Weight Cycle

Using Bellman-Ford Algorithm

def detect_negative_cycle(vertices, edges):

Initialize distances

distance = [0] * vertices

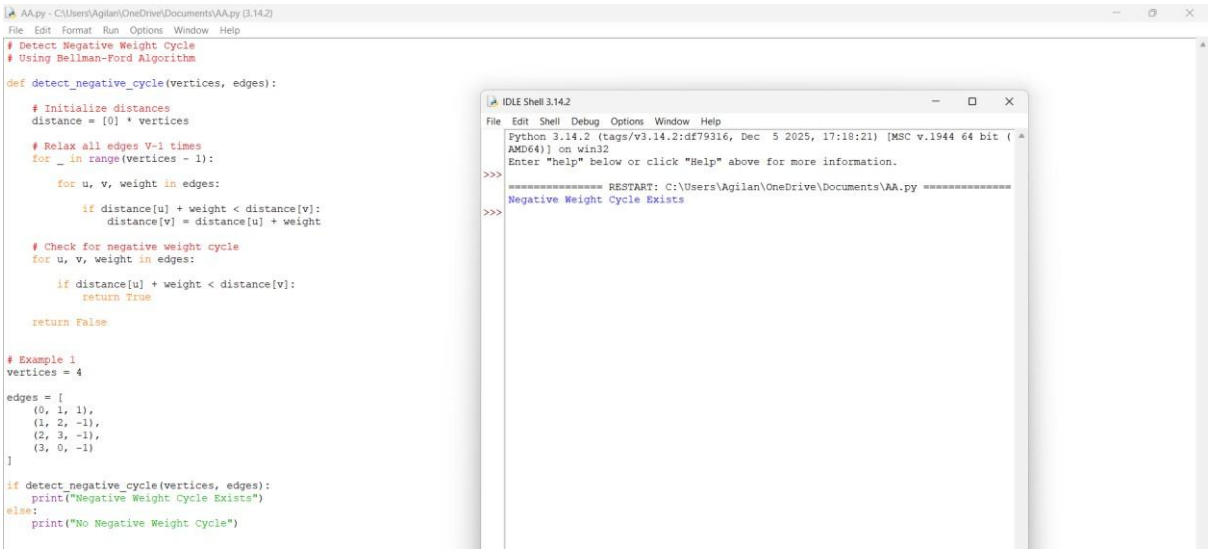
```
    # Relax all edges V-1 times
for _ in range(vertices - 1):

        for u, v, weight in edges:

            if distance[u] + weight < distance[v]:
distance[v] = distance[u] + weight

    # Check for negative weight cycle
for u, v, weight in edges:

        if distance[u] + weight < distance[v]:
            return True

    return False



# Example 1
vertices = 4

edges = [
    (0, 1, 1),
    (1, 2, -1),
    (2, 3, -1),
    (3, 0, -1)
]
```

if detect_negative_cycle(vertices, edges):

print("Negative Weight Cycle Exists") else:

print("No Negative Weight Cycle")

EXECUTION & OUTPUT:



DETERMINE RECHABILITY BETWEEN TWO VERTICES

PYTHON CODE:

Determine Reachability Between Two Vertices

Using Warshall's Algorithm

def is_reachable(graph, u, v):

n = len(graph)

Create a copy of the graph

reach = [row[:] for row in graph]

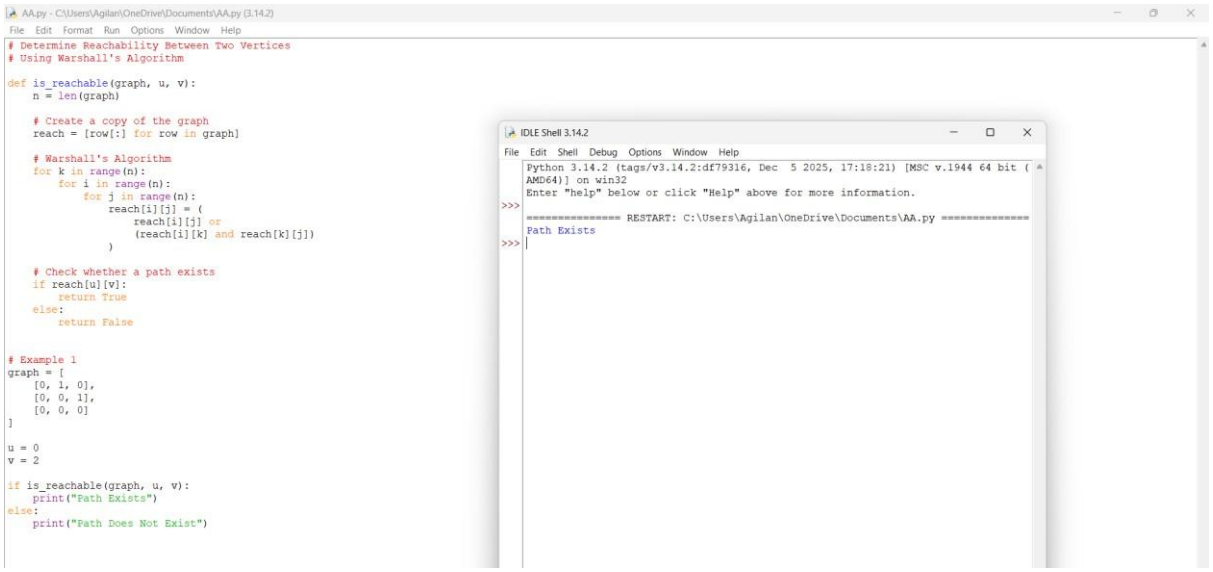
Warshall's Algorithm

for k in range(n):

for i in range(n):

```
for j in range(n):
reach[i][j] = (
reach[i][j] or
                (reach[i][k] and reach[k][j])
            )

    # Check whether a path exists
if reach[u][v]:          return True
else:        return False



# Example 1 graph
= [
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0]
]

u = 0
v = 2

if is_reachable(graph, u, v):
    print("Path Exists") else:
    print("Path Does Not Exist")
```

EXECUTION & OUTPUT:



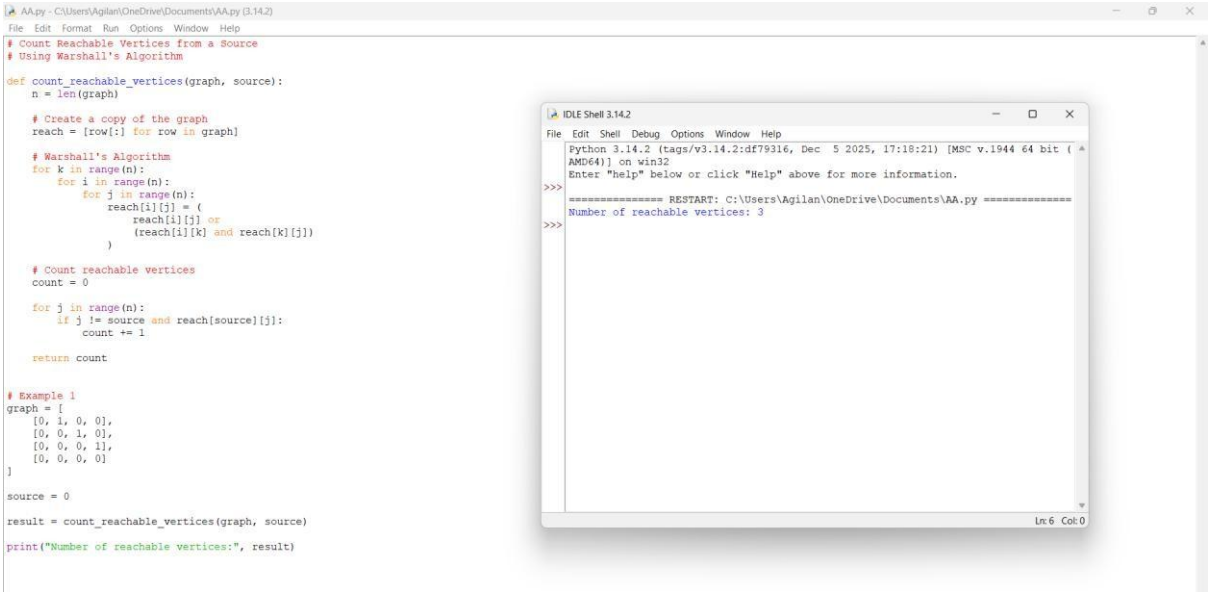
COUNT REACHABLE VERTICES FROM A SOURCE

PYTHON CODE:

```
# Count Reachable Vertices from a Source
# Using Warshall's Algorithm


def count_reachable_vertices(graph, source):
    n = len(graph)


    # Create a copy of the graph
    reach = [row[:] for row in graph]


    # Warshall's Algorithm
for k in range(n):
for i in range(n):
for j in range(n):
reach[i][j] = (
reach[i][j] or
                (reach[i][k] and reach[k][j])
            )
```

```
    # Count reachable vertices
count = 0

    for j in range(n):          if j != source
and reach[source][j]:
            count += 1

    return count



# Example 1 graph
= [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

source = 0

result = count_reachable_vertices(graph, source)

print("Number of reachable vertices:", result)
```

EXECUTION & OUTPUT:



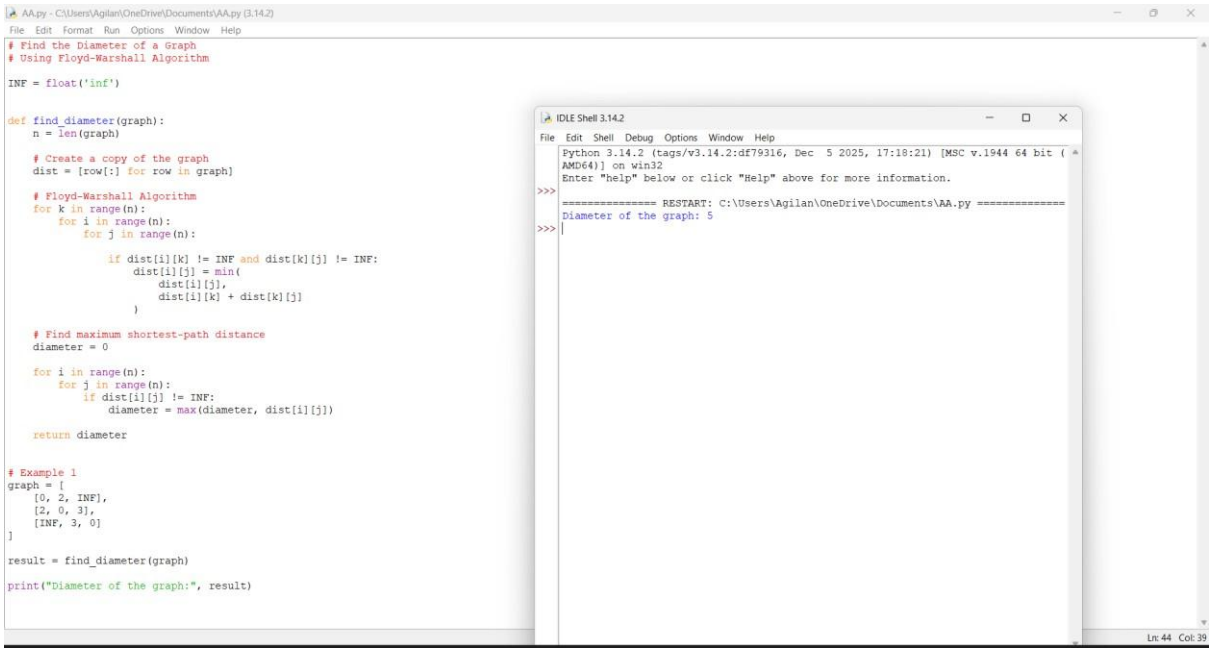
FIND THE DIAMETER OF A GRAPH

PYTHON CODE:

```
# Find the Diameter of a Graph
# Using Floyd-Warshall Algorithm

INF = float('inf')


def find_diameter(graph):
    n = len(graph)

    # Create a copy of the graph
dist = [row[:] for row in graph]

    # Floyd-Warshall Algorithm
for k in range(n):          for i in
range(n):               for j in
range(n):
```

```
            if dist[i][k] != INF and dist[k][j] != INF:
                dist[i][j] = min(
dist[i][j],                     dist[i][k]
+ dist[k][j]
                )

    # Find maximum shortest-path distance
diameter = 0

    for i in range(n):
for j in range(n):
if dist[i][j] != INF:
            diameter = max(diameter, dist[i][j])

    return diameter



# Example 1 graph
= [
    [0, 2, INF],
    [2, 0, 3],
    [INF, 3, 0]
]

result = find_diameter(graph)

print("Diameter of the graph:", result)
```

EXECUTION & OUTPUT:



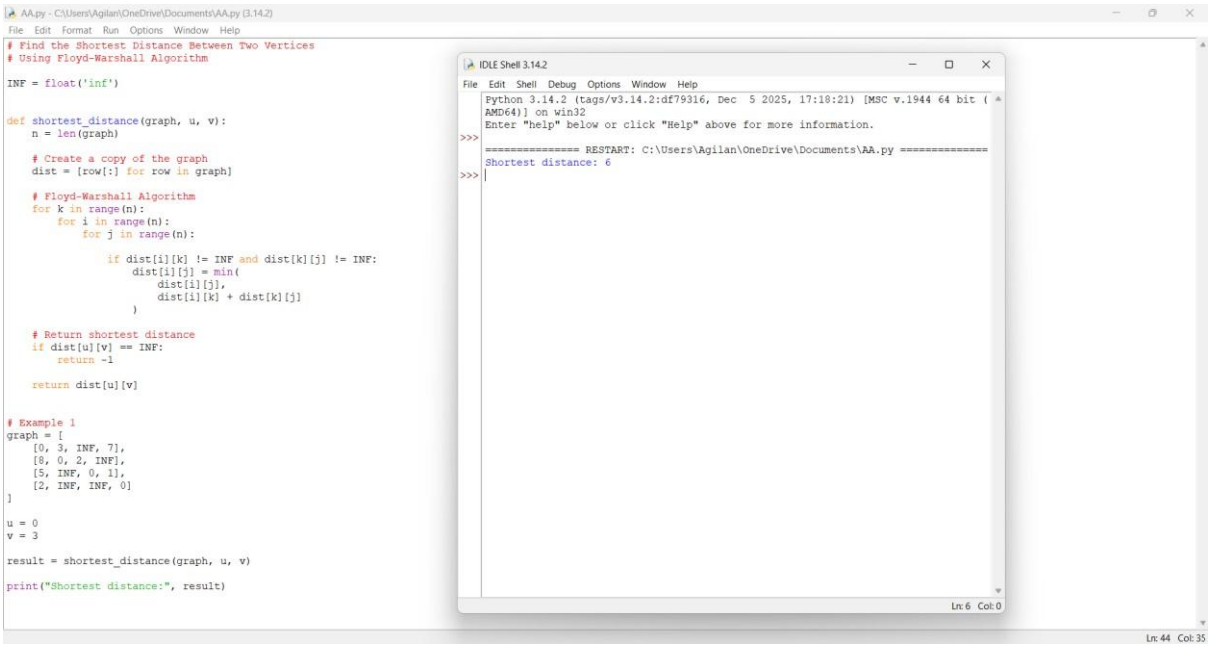
FIND THE SHORTEST DISTANCE BETWEEN TWO VERTICES

PYTHON CODE:

```
# Find the Shortest Distance Between Two Vertices
# Using Floyd-Warshall Algorithm


INF = float('inf')



def shortest_distance(graph, u, v):
    n = len(graph)


    # Create a copy of the graph
dist = [row[:] for row in graph]


    # Floyd-Warshall Algorithm
for k in range(n):          for i in
range(n):               for j in
range(n):
```

```
            if dist[i][k] != INF and dist[k][j] != INF:
                dist[i][j] = min(
dist[i][j],                     dist[i][k]
+ dist[k][j]
                )

    # Return shortest distance
if dist[u][v] == INF:
        return -1

    return dist[u][v]



# Example 1 graph
= [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]

u = 0
v = 3

result = shortest_distance(graph, u, v)

print("Shortest distance:", result)
```

EXECUTION & OUTPUT:



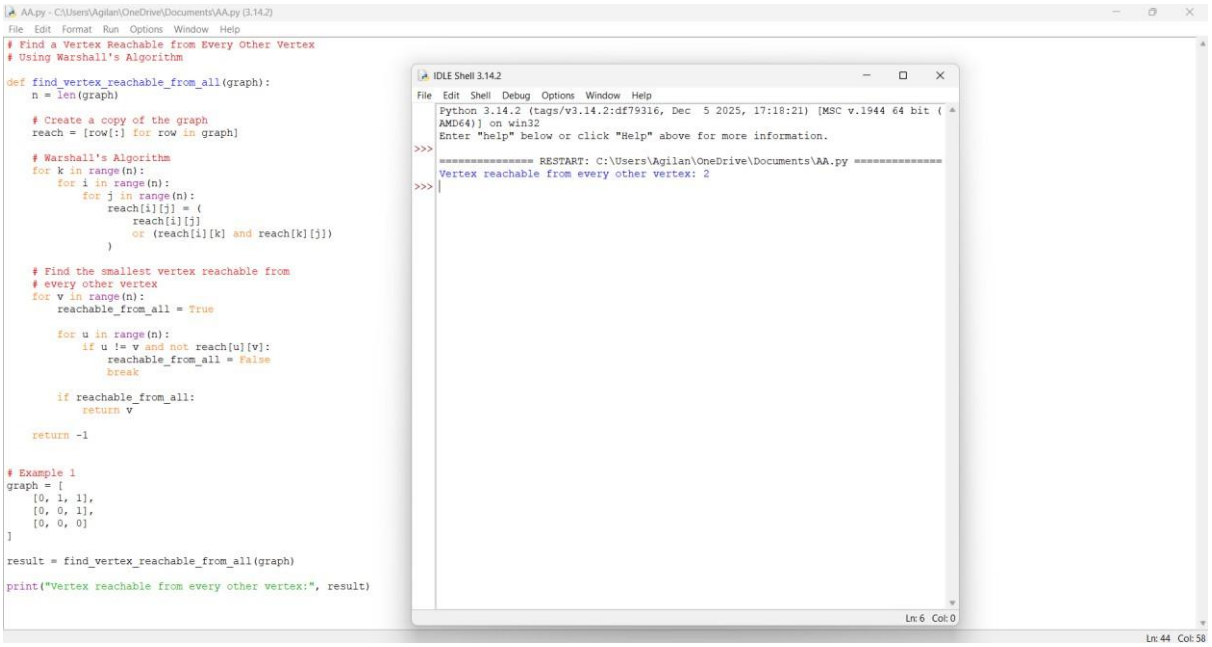
FIND A VERTEX REACHABLE FROM EVERY OTHER VERTEX

PYTHON CODE:

```
# Find a Vertex Reachable from Every Other Vertex
# Using Warshall's Algorithm


def find_vertex_reachable_from_all(graph):
    n = len(graph)


    # Create a copy of the graph
    reach = [row[:] for row in graph]


    # Warshall's Algorithm     for k in
range(n):         for i in range(n):             for
j in range(n):                 reach[i][j] = (
reach[i][j]                 or (reach[i][k] and
reach[k][j])
                )
```

```
    # Find the smallest vertex reachable from
    # every other vertex
for v in range(n):
        reachable_from_all = True

        for u in range(n):              if u !=
v and not reach[u][v]:
reachable_from_all = False
break

        if reachable_from_all:
            return v

    return -1



# Example 1
graph = [
    [0, 1, 1],
    [0, 0, 1],
    [0, 0, 0]
]

result = find_vertex_reachable_from_all(graph)

print("Vertex reachable from every other vertex:", result)
```

EXECUTION & OUTPUT:



FIND THE VERTEX WITH MAXIMUM REACHZBILITY

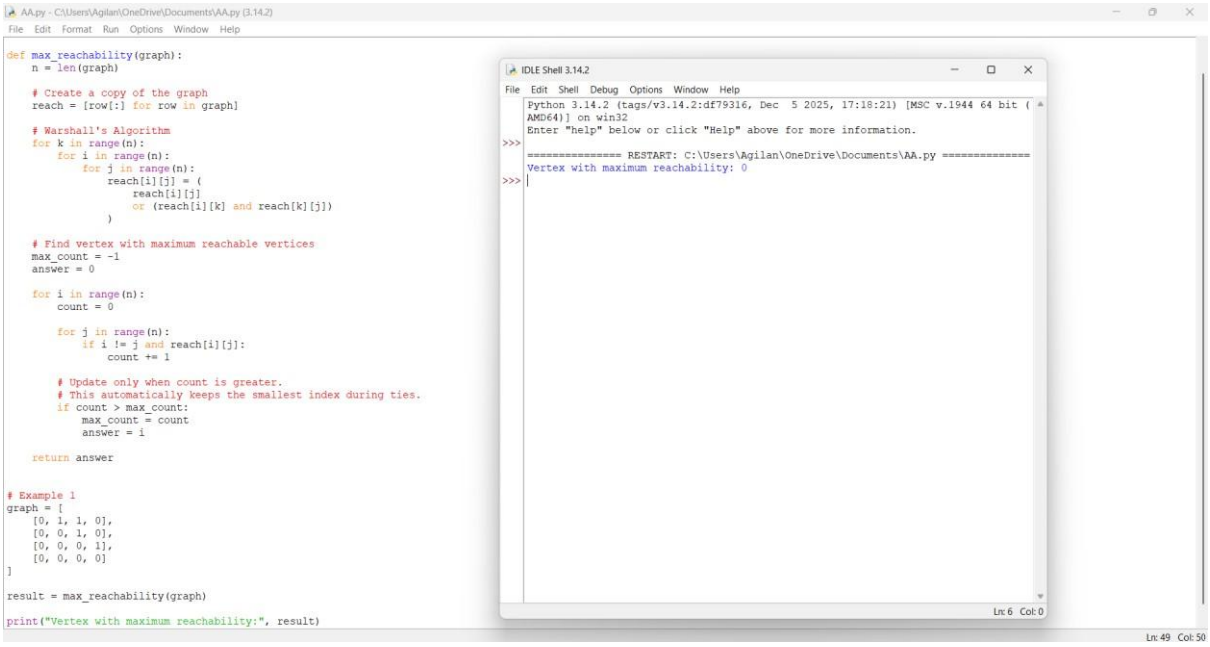
PYTHON CODE:

```
# Find the Vertex with Maximum Reachability
# Using Warshall's Algorithm


def max_reachability(graph):
    n = len(graph)

    # Create a copy of the graph
reach = [row[:] for row in graph]

    # Warshall's Algorithm     for k in
range(n):         for i in range(n):              for
j in range(n):                  reach[i][j] = (
reach[i][j]                     or (reach[i][k] and
reach[k][j])
                )
```

```
    # Find vertex with maximum reachable vertices
max_count = -1      answer = 0

    for i in range(n):
count = 0

        for j in range(n):
if i != j and reach[i][j]:
                count += 1

        # Update only when count is greater.
        # This automatically keeps the smallest index during ties.
if count > max_count:               max_count = count
answer = i

    return answer



# Example 1 graph
= [
    [0, 1, 1, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

result = max_reachability(graph)
```

print("Vertex with maximum reachability:",
result)

EXECUTION & OUTPUT:



FIND THE DIAMETER OF A WEIGHTED GRAPH

PYTHON CODE:

Find the Diameter of a Weighted Graph

Using Floyd-Warshall Algorithm

INF = float('inf')

def find_diameter(graph):

n = len(graph)

Create a copy of the graph

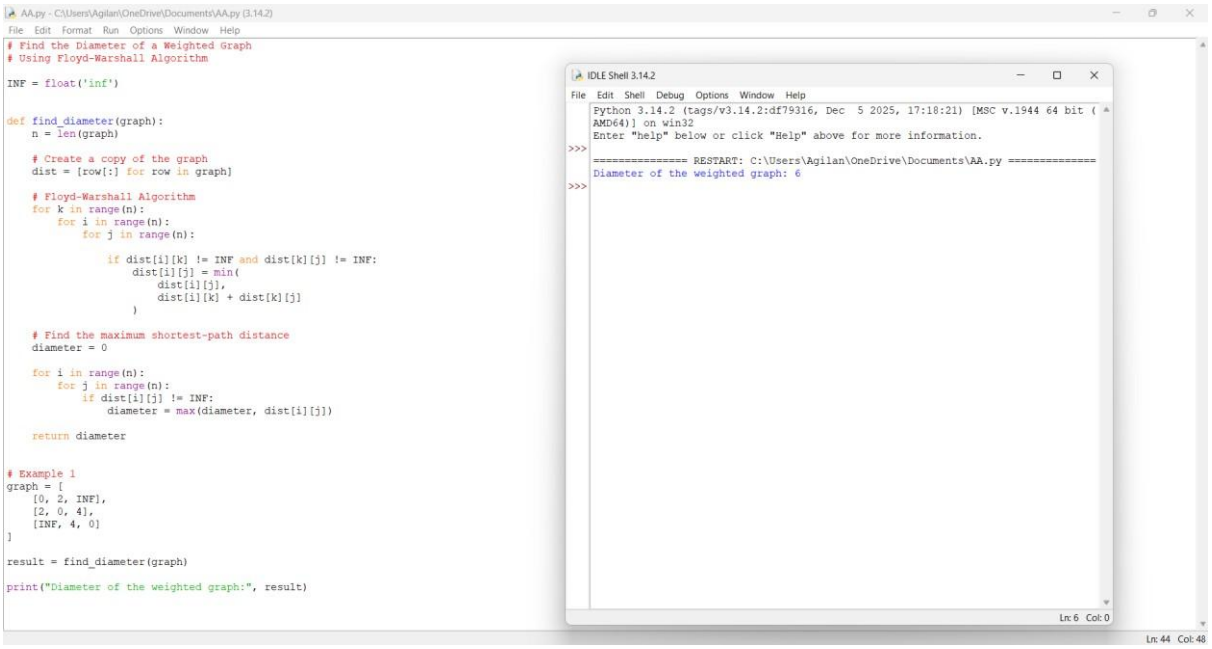
dist = [row[:] for row in graph]

```
    # Floyd-Warshall Algorithm
for k in range(n):         for i in
range(n):              for j in
range(n):


                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
dist[i][j],                         dist[i][k]
+ dist[k][j]
                    )


    # Find the maximum shortest-path distance
diameter = 0


    for i in range(n):
for j in range(n):
if dist[i][j] != INF:
diameter = max(diameter,
dist[i][j])


    return diameter



# Example 1 graph
= [
    [0, 2, INF],
    [2, 0, 4],
    [INF, 4, 0]
]
```

result = find_diameter(graph)

print("Diameter of the weighted graph:", result)

EXECUTION & OUTPUT:



CHECK WHETHER A GRAPH IS STRONGLY CONNECTED

PYTHON CODE:

Check Whether a Graph is Strongly Connected

Using Warshall's Algorithm

def is_strongly_connected(graph):

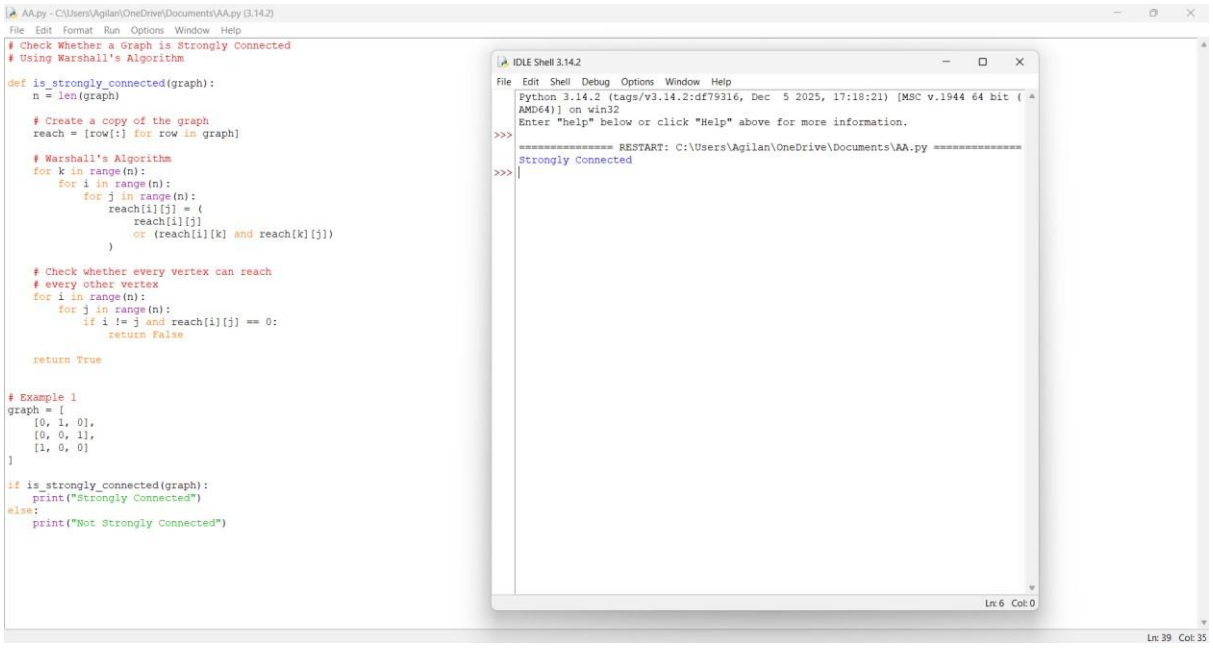
n = len(graph)

Create a copy of the graph

reach = [row[:] for row in graph]

```
    # Warshall's Algorithm     for k in
range(n):         for i in range(n):             for
j in range(n):                 reach[i][j] = (
reach[i][j]                   or (reach[i][k] and
reach[k][j])
                )

    # Check whether every vertex can reach
    # every other vertex     for i in
range(n):         for j in range(n):
if i != j and reach[i][j] == 0:
                return False

    return True


# Example 1 graph
= [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]

if is_strongly_connected(graph):
print("Strongly Connected") else:
    print("Not Strongly Connected")
```

EXECUTION & OUTPUT:


AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)
File Edit Format Run Options Window Help
# Check Whether a Graph is Strongly Connected
# Using Warshall's Algorithm
def is_strongly_connected(graph):
n = len(graph)
# Create a copy of the graph
reach = [row[:] for row in graph]
# Warshall's Algorithm
for k in range(n):
for i in range(n):
for j in range(n):
reach[i][j] = (
reach[i][j]
or (reach[i][k] and reach[k][j])
)
# Check whether every vertex can reach
# every other vertex
for i in range(n):
for j in range(n):
if i != j and reach[i][j] == 0:
return False
return True
# Example 1
graph = [
[0, 1, 0],
[0, 0, 1],
[1, 0, 0]
]
if is_strongly_connected(graph):
print("Strongly Connected")
else:
print("Not Strongly Connected")
Ln: 39 Col: 35
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\Agilan\OneDrive\Documents\AA.py ==========
Strongly Connected
>>>
Ln: 6 Col: 0